VERIQTA

LEARN
PRACTICE
TROUBLESHOOT
GROW

Better Engineers Build A Safer Tomorrow

THE JUNIOR DEVOPS ENGINEER'S SURVIVAL GUIDE

SMALL STEPS BIG PROGRESS

TOOLS SKILLS PRACTICE REAL-WORLD CONFIDENCE

From Your First Day to Confident Production Work

LINUX · GIT · DOCKER · KUBERNETES · AWS · TERRAFORM

CI/CD · MONITORING · SECURITY · CLOUD · INCIDENTS · AUTOMATION

- ☑ PRACTICAL GUIDANCE
- ☑ REAL-WORLD EXAMPLES
- ☑ COMMON MISTAKES TO AVOID
- ☑ TROUBLESHOOTING STEPS
- ☑ PRODUCTION BEST PRACTICES

SAME PROBLEMS BETTER ENGINEERS

UNDERSTAND
PRACTICE
TROUBLESHOOT
DEPLOY
GROW

YOUR DEVOPS JOURNEY STARTS HERE

BUILD | DEPLOY | OPERATE | IMPROVE

@veriqta | veriqta | veriqta | veriqta | @veriqta | @veriqta

VERIQTA

Learn
Build
Deploy
Grow

Same
Problems
Stronger
Engineers

WELCOME TO DEVOPS,
WHAT YOU ACTUALLY SIGNED UP FOR

IT'S A JOURNEY, NOT JUST A JOB TITLE

1 WHAT A JUNIOR DEVOPS ENGINEER REALLY DOES

- Works with infrastructure, CI/CD, deployment, monitoring, and automation
- Supports application teams in delivering software
- Troubleshoots issues in development, staging and production
- Learns and improves existing systems
- Collaborates with developers, security, and operations
- Helps keep systems reliable, secure, and scalable

2 DEVOPS IS MORE THAN CI/CD

CI/CD is important, but DevOps is much bigger.

INFRASTRUCTURE
Servers, Cloud, Networking

AUTOMATION
Scripting, IaC, Tools

DEPLOYMENT
Releasing software safely

RELIABILITY
Monitoring, Incident response

OPERATIONS
Keeping systems running

SECURITY
Following best practices

3 INFRASTRUCTURE, AUTOMATION, DEPLOYMENT, RELIABILITY, AND OPERATIONS

As a DevOps engineer, you work across the entire software lifecycle:

BUILD (Developer) → DEPLOY (DevOps) → RUN (Operations) → IMPROVE (Everyone)

DELIVER BETTER SOFTWARE, FASTER AND SAFER

4 WHAT YOU ARE EXPECTED TO KNOW

- Linux basics
- Networking fundamentals
- Version control (Git)
- Containers (Docker)
- Kubernetes basics
- CI/CD concepts
- Cloud basics (AWS, Azure, or GCP)
- How to read logs and troubleshoot
- Willingness to learn

5 WHAT YOU ARE NOT EXPECTED TO KNOW YET

- You are not expected to know everything
- You are not expected to be an expert in all tools
- You are not expected to solve complex incidents alone
- You are not expected to know the entire architecture
- You are not expected to have years of experience
- You are expected to ask questions and learn

Everyone was a beginner at some point.

6 THE MOST IMPORTANT SKILL

KNOWING HOW TO INVESTIGATE.

- Read error messages
- Check logs
- Understand what changed
- Use documentation
- Try, test, and verify
- Ask the right questions
- Learn from mistakes

You don't need to know everything.
You need to know how to find the answer.

7 JUNIOR ENGINEER TIP

You do not need to memorize every tool.

Focus on understanding concepts, how things work, and how to troubleshoot.
With time and practice, the tools will become familiar.

Progress
Over
Perfection

SAME PROBLEMS. BETTER ENGINEERS.

VERIQTA
LEARN | PRACTICE | DEPLOY | GROW

 @veriqta |  veriqta |  veriqta |  veriqta |  @veriqta |  @veriqta

VERIQTA

GOOD ENGINEERS LEARN FIRST THEN CHANGE

PAGE 2

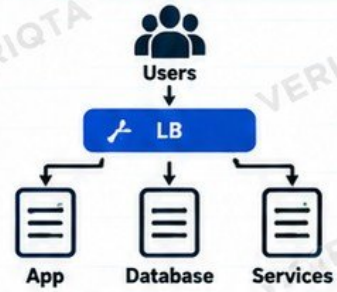
YOUR FIRST WEEK

UNDERSTAND BEFORE YOU CHANGE

SAME PROBLEMS STRONGER ENGINEERS

OBSERVE. LEARN. ASK. THEN BUILD.

1 LEARN THE TEAM'S ARCHITECTURE



- Understand how the system works
- Identify key components
- Learn data flow

You don't need to know everything. You just need to understand the big picture.

2 IDENTIFY ENVIRONMENTS

DEV	Development (Testing, daily work)
TEST	Testing environment (QA, validation)
STAGING	Pre-production (Like production)
PRODUCTION	Live environment (Real users)

- Learn the purpose of each environment
- Understand how changes move between them
- Find out who can deploy to each

3 FIND REPOSITORIES AND DOCUMENTATION

- Locate source code repositories
- Read README files
- Find infrastructure as code (IaC)
- Find deployment configurations
- Read team documentation (wikis, runbooks, diagrams)

Documentation is your best friend. Read it before asking questions.

4 UNDERSTAND WHO OWNS WHAT

- Know the team structure
- Identify service owners
- Understand who to contact for different systems
- Learn about external teams (Dev, QA, Security, Network, etc.)

Knowing the right person saves time and prevents mistakes.

5 LEARN COMMUNICATION AND ESCALATION CHANNELS

Slack (Team chat) · Teams (Communication) · Email (Formal updates)

- Learn how the team communicates
- Know when and how to escalate
- Understand on-call rotation
- Find out the incident process
- Use clear and professional communication

6 FIND DASHBBORIES AND RUNBOOKS

- Locate monitoring dashboards (e.g. Grafana, Datadog, CloudWatch)
- Find log sources
- Read runbooks for common issues
- Understand alerting and notifications
- Explore existing incident reviews

Dashboards help you see problems early. Runbooks help you respond correctly.

7 UNDERSTAND ACCESS BOUNDARIES

- Learn what you have access to
- Understand permission levels
- Know what requires approval
- Never share your credentials
- Follow security policies (MFA, VPN, etc.)

Access is a responsibility. Use it carefully.

8 NEVER MAKE PRODUCTION CHANGES JUST TO "SEE WHAT HAPPENS"

- Do not experiment in production
- Follow change management processes
- Always test in lower environments
- Understand the impact before making changes
- Ask for guidance when unsure

Curiosity is good. Uncontrolled changes can cause outages, data loss, and loss of trust.

9 FIRST-WEEK CHECKLIST

- [x] Understand the architecture
- [x] Identify all environments
- [x] Find repositories and documentation
- [x] Know team members and owners
- [x] Set up communication channels
- [x] Locate dashboards and runbooks
- [x] Understand your access boundaries
- [x] Read ongoing tickets and projects
- [x] Ask questions and take notes
- [x] Do not make production changes

Learn → Observe → Ask → Contribute

A STRONG START LEADS TO A SUCCESSFUL JOURNEY.

VERIQTA

LEARN | PRACTICE | DEPLOY | GROW

@veriqta | veriqta | veriqta | veriqta | @veriqta | @veriqta

PROGRESS OVER PERFECTION

LEARN TO READ THE SYSTEM

SAME PROBLEMS STRONGER ENGINEERS

UNDERSTAND HOW EVERYTHING CONNECTS

1 APPLICATION

- The software users interact with.
- Can be a web app, API, service or microservice.
- Runs on servers, containers, or Kubernetes.

2 SOURCE REPOSITORY

- Stores the application code.
- Usually Git (GitHub, GitLab, Bitbucket, etc.).
- Tracks changes and enables collaboration.

3 CI/CD PIPELINE

- Builds, tests, and packages the code.
- Runs automated checks (security, quality, etc.).
- Deploys the application to the target environment.

4 ARTIFACT OR CONTAINER REGISTRY

- Stores build artifacts like container images or packages.
- Examples: Docker Hub, ECR, GCR, Artifactory, Nexus.
- Provides versioned, immutable artifacts for deployment.

5 INFRASTRUCTURE

- The underlying resources that run your application.
- Can be cloud (AWS, Azure, GCP) or on-premises.
- Includes compute, storage, network, and more.

6 KUBERNETES OR COMPUTE PLATFORM

- Orchestrates containers and manages application deployment.
- Handles scaling, self-healing, and high availability.
- Alternatives include VMs, serverless, or other compute platforms.

7 DATABASE

- Stores application data.
- Examples: PostgreSQL, MySQL, MongoDB, RDS, etc.
- Requires backups, monitoring, and access control.

8 NETWORKING

- Enables communication between components.
- Includes VPCs, subnets, load balancers, DNS, firewalls, and ingress.
- Controls traffic flow and security boundaries.

9 OBSERVABILITY

- Helps you see what is happening in the system.
- Includes logs, metrics, and traces.
- Examples: Prometheus, Grafana, ELK, Datadog, Dynatrace.
- Used for monitoring, alerting, and troubleshooting.

ARCHITECTURE FLOW

FROM CODE TO USERS

Developer — Writes and commits code

→ **Git** — Source repository (e.g. GitHub)

→ **CI/CD Pipeline** — Build, test and deploy

→ **Artifact** — Container registry (e.g. ECR, Docker Hub)

→ **Infrastructure** — Cloud or on-prem resources

→ **Application** — Runs on Kubernetes or compute platform

→ **Users** — Access the application

JUNIOR ENGINEER TIP

Always understand how the pieces fit together.
It helps you troubleshoot faster and makes you a more effective engineer.

OBSERVE UNDERSTAND TROUBLESHOOT IMPROVE

BETTER ENGINEERS A BRIGHTER TOMORROW

VERIQTA

PRACTICE
EXPLORE
LEARN
IMPROVE

PAGE 4

LINUX SURVIVAL SKILLS

YOU WILL ACTUALLY USE

SMALL STEPS BIG PROGRESS

SAME PROBLEMS STRONGER ENGINEERS

REAL COMMANDS. REAL EXAMPLES. REAL-WORLD SKILLS.

1 NAVIGATING SERVERS

Move around the filesystem.

```
pwd       # show current directory
ls        # list files and directories
cd        # change directory
cd ..     # go up one directory
cd /      # go to root directory
cd ~      # go to home directory
```

2 FILES AND DIRECTORIES

Work with files and directories.

```
ls -l             # detailed list
ls -lh            # human readable size
mkdir mydir       # create directory
touch file.txt    # create file
rm file.txt       # delete file
rm -rf mydir      # delete directory
cp file1 file2    # copy file
mv file1 file2    # move/rename file
```

3 VIEWING FILE CONTENT

Read and inspect files.

```
cat file.txt          # show entire file
less file.txt         # view file page by page
tail file.txt         # show last 10 lines
tail -n 50 file.txt   # last 50 lines
head file.txt         # show first 10 lines
head -n 50 file.txt   # first 50 lines
```

4 SEARCHING WITH GREP

Find text inside files.

```
grep "error" app.log       # search for text
grep -i "error" app.log    # case insensitive
grep -r "error" /var/log   # search recursively
grep -n "error" app.log    # show line numbers
```

5 FINDING FILES

Search for files and directories.

```
find / -name "app.log"          # find file by name
find / -type d -name "logs"     # find directory
find . -name "*.txt"            # find by pattern
find / -size +100M              # find large files
```

6 PROCESS MANAGEMENT

View and manage running processes.

```
ps aux                # list all processes
ps -ef                # full format
ps aux | grep nginx   # find specific process
top                   # real-time process monitor
htop                  # (if installed) better view
kill <PID>            # stop a process
kill -9 <PID>         # force kill a process
```

7 DISK USAGE

Check disk space and usage.

```
df -h                 # disk space usage
df -hT                # show filesystem type
du -sh /var/log       # size of directory
du -h --max-depth=1 /     # size of folders
```

8 MEMORY USAGE

Check memory usage.

```
free -h               # memory usage (human readable)
free -m               # in MB
cat /proc/meminfo     # detailed memory info
top                   # live memory usage (press M)
```

9 SYSTEMD (SERVICE MANAGEMENT)

Manage services on Linux.

```
systemctl status nginx    # check service status
systemctl start nginx     # start service
systemctl stop nginx      # stop service
systemctl restart nginx   # restart service
systemctl enable nginx    # start on boot
systemctl disable nginx   # disable on boot
systemctl list-units --type=service
                          # list all services
```

10 READING LOGS WITH JOURNALCTL

View system and service logs.

```
journalctl -xe            # view recent logs (with details)
journalctl -u nginx       # logs for a specific service
journalctl -u nginx -f    # follow logs in real time
journalctl --since "1 hour ago"    # logs from last hour
journalctl --until "2024-01-01 10:00"  # logs until a time
```

11 WHY SUDO SHOULD NOT BE YOUR DEFAULT SOLUTION

- sudo gives elevated (root) privileges.
- It can make accidental changes that break systems.
- It bypasses normal permission boundaries.
- It can hide the real problem instead of fixing it.
- Use sudo only when necessary and understand the command.
- Learn proper user permissions and least privilege.
- Always know what a command does before running it with sudo.

BAD HABITS TO AVOID

- ✗ sudo for every command
- ✗ Running commands you don't understand
- ✗ Making changes directly on production
- ✗ Ignoring warnings or errors
- ✗ Copying random commands from the internet

JUNIOR ENGINEER TIP

You do not need to memorize every command.
Focus on understanding what each command does, and practice using them.
With time, they will become natural.

PRACTICE TODAY
A STRONGER ENGINEER
TOMORROW

GOOD NETWORKS KEEP BUSINESS RUNNING

SAME PROBLEMS STRONGER ENGINEERS

NETWORKING SURVIVAL FOR DEVOPS ENGINEERS

LEARN EXPLORE PRACTICE IMPROVE

UNDERSTAND. TROUBLESHOOT. KEEP SYSTEMS CONNECTED.

1 IP ADDRESSES

An IP address uniquely identifies a device on a network.

- IPv4 example: 192.168.1.10
- IPv6 example: 2001:db8::1
- Can be public or private
- Used for routing and communication

2 DNS (DOMAIN NAME SYSTEM)

DNS converts domain names to IP addresses.

- Example: www.google.com → 142.250.72.14
- Use dig or nslookup to query DNS
- DNS records: A, AAAA, CNAME, MX, TXT
- If DNS fails, you cannot reach the service

3 PORTS

Ports identify specific services on a device.

- Common ports: 80 (HTTP), 443 (HTTPS), 22 (SSH), 53 (DNS)
- Ports can be open or closed
- Use ss or netstat to check open ports
- Firewalls control which ports are accessible

4 TCP (TRANSMISSION CONTROL PROTOCOL)

TCP provides reliable, ordered, and error-checked delivery of data.

- Used by most applications (HTTP, SSH, etc.)
- Establishes a connection (three-way handshake)
- Ensures data is delivered in the correct order
- If a packet is lost, TCP retransmits it

5 HTTP AND HTTPS

HTTP is used to transfer data between a client and server. HTTPS adds encryption (TLS).

- HTTP uses port 80
- HTTPS uses port 443
- HTTPS encrypts data for security
- Use curl to test web services
- Check HTTP status codes (200, 301, 404, 500)

6 ROUTING

Routing determines the path traffic takes to reach a destination.

- Uses routing tables
- Traffic can go through multiple hops
- In the cloud, routers and route tables control traffic between subnets, VPCs, and the internet
- Incorrect routes can cause connectivity issues

7 FIREWALLS

Firewalls allow or block traffic based on rules.

- Can be host-based or network-based
- Used to control inbound and outbound traffic
- Common in cloud (security groups, NACLs) and on-premises
- Misconfigured firewalls can block legitimate traffic

8 CURL

curl is a command-line tool for making HTTP requests.

```
curl http://example.com      # basic request
curl -I https://example.com  # check headers
curl -v https://example.com  # verbose output
```

- Useful for testing APIs and web services
- Helps check response codes, headers, and latency

9 PING

ping tests basic network connectivity using ICMP.

```
ping example.com
```

- Checks if a host is reachable
- Shows round-trip time (latency)
- Does not test application ports
- May be blocked by firewalls

10 DIG

dig is a powerful tool to query DNS records.

```
dig example.com       # query A record
dig example.com MX    # query MX record
dig -x 8.8.8.8        # reverse lookup
```

- Useful for troubleshooting DNS issues
- Shows detailed DNS response information

11 SS

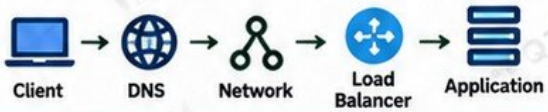
ss shows network connections, listening ports, and socket statistics.

```
ss -tuln      # show listening ports
ss -t         # show TCP connections
ss -u         # show UDP connections
```

- Modern alternative to netstat
- Helps verify what services are listening

12 MENTAL MODEL

How a request flows from a user to an application.



Client → DNS → Network → Load Balancer → Application

Each layer must work for the request to succeed.

13 TROUBLESHOOTING "CONNECTION REFUSED" VS "TIMEOUT"

CONNECTION REFUSED

- The host is reachable, but the port is closed.
- The service is not running or not listening on that port.
- You will usually get a quick response.
- Check if the application is running and listening.
- Verify firewall rules and the correct port.

Example: curl: (7) Failed to connect: Connection refused

TIMEOUT

- The request did not get a response within the expected time.
- The host may be unreachable or traffic is being blocked.
- Often caused by network issues, firewalls, or incorrect routing.
- It takes longer to fail (waits for a timeout).
- Check network connectivity, firewall rules, and routing.

Example: curl: (28) Operation timed out

JUNIOR ENGINEER TIP

When something doesn't work, break it down: Is it DNS, network, firewall, or the application? Test each layer.

NETWORK KNOWLEDGE TURNS GOOD ENGINEERS INTO GREAT ONES.

GOOD COMMITS BETTER SOFTWARE

GIT WITHOUT DESTROYING THE TEAM REPOSITORY

SAME PROBLEMS STRONGER ENGINEERS

LEARN EXPLORE PRACTICE IMPROVE

COLLABORATE. CONTRIBUTE. DON'T BREAK THINGS.

1 CLONE

Get a copy of a repository on your local machine.

```
git clone https://github.com/example/repo.git
cd repo
```

- Downloads the repository and its history.
- Use SSH or HTTPS.

2 PULL

Get the latest changes from the remote repository.

```
git pull origin main
```

- Fetches and merges changes into your current branch.
- Run this regularly to stay updated.

3 BRANCH

Create a new branch for your work.

```
git branch feature/login
git checkout feature/login
```

- Keep your work separate.
- Use meaningful branch names (e.g. feature/login, fix/bug).

4 STATUS

Check the current status of your repository.

```
git status
```

- Shows changed files, untracked files, and branch info.
- Helps you know what will be committed.

5 ADD

Stage files for commit.

```
git add file.txt
git add .
```

- Adds specific file or all changes to the staging area.
- Staging allows you to review changes before committing.

6 COMMIT

Save your changes with a message.

```
git commit -m "Add login page"
```

- Write clear and meaningful commit messages.
- Describe what and why, not just what.

7 PUSH

Upload your changes to the remote repository.

```
git push origin feature/login
```

- Pushes your local commits to the remote branch.
- Use the correct branch name.

8 PULL REQUESTS

Propose your changes for review and merge.

- Open a pull request (PR) on GitHub, GitLab, or Bitbucket.
- Describe what you changed and why.
- Request review from team members.
- Do not merge without approval if your team requires it.

9 MERGE CONFLICTS

Happens when changes conflict between branches.

- Git cannot automatically merge the changes.
- Edit the conflicting files, resolve the differences, then commit.
- Use a merge tool or your editor.
- Test after resolving conflicts.

10 WHY FORCE-PUSHING CAN BE DANGEROUS

- `git push --force` overwrites history on the remote branch.
- It can delete other people's work.
- Only use force push on your own branch and when you are sure.
- Use `git push --force-with-lease` for a safer option.

Always think before you force push. It can break the team's work.

11 NEVER COMMIT CREDENTIALS

Do not commit sensitive information to Git.

- Never commit passwords, API keys, tokens, or private keys.
- Use environment variables or secret management tools (e.g. GitHub Secrets, Vault).
- If you accidentally commit credentials, remove them immediately and rotate the keys.

12 BRANCHES HAVE DIVERGED?

When Git says your branches have diverged, it means both your local and remote branches have new commits.

SOLUTIONS:

Option 1: Pull and merge

```
git pull origin main
```

Option 2: Pull and rebase

```
git pull --rebase origin main
```

Resolve any conflicts, then push.

JUNIOR ENGINEER TIP

Commit small, logical changes.
Use clear messages. When in doubt, ask.
Git is powerful, but your teammates' trust is more important.

"GOOD VERSION CONTROL CREATES BETTER ENGINEERS AND STRONGER TEAMS."

PRACTICE TODAY A STRONGER ENGINEER TOMORROW

PLAN EXECUTE LEARN IMPROVE

SAME PROBLEMS STRONGER ENGINEERS

TICKETS, TASKS, AND YOUR FIRST ENGINEERING CHANGE

GOOD CHANGES BUILD GREAT CAREERS

BEFORE YOU CHANGE. UNDERSTAND. PLAN. EXECUTE. VERIFY.

1 UNDERSTAND THE REQUIREMENT BEFORE TOUCHING INFRASTRUCTURE

- Read the ticket carefully.
- Understand the problem and the expected outcome.
- Ask questions if anything is unclear.
- Do not make assumptions.

"A clear understanding prevents mistakes and rework."

2 READ ACCEPTANCE CRITERIA

- Check what "done" looks like.
- Identify functional and non-functional requirements.
- Look for success criteria, testing steps, and rollback plan.
- Confirm with the requester if needed.

"If you don't know what success looks like, you might never reach it."

3 IDENTIFY AFFECTED SYSTEMS

- Find out which services, environments, and teams are affected.
- Check dependencies (databases, APIs, other services).
- Understand the potential impact (blast radius).

"Know what you can break before you make a change."

4 FIND THE RELEVANT REPOSITORY

- Locate the correct code or infrastructure repository.
- Check the README and documentation.
- Understand the project structure.
- If unsure, ask the team.

"Work in the right place."

5 CHECK EXISTING PATTERNS

- Look at previous changes and pull requests.
- Follow the team's standards and conventions.
- Reuse existing modules or templates.
- Keep your change consistent with the codebase.

"Don't reinvent the wheel."

6 MAKE THE SMALLEST SAFE CHANGE

- Change only what is necessary.
- Keep the scope small and focused.
- Avoid unrelated changes in the same pull request.
- Smaller changes are easier to review, test, and roll back.

"Small, safe changes build trust."

7 TEST IT

- Run local tests if available.
- Validate your changes in a non-production environment (dev/test).
- Check that existing functionality still works.
- Test edge cases where possible.

"Test early. Test properly."

8 OPEN A PULL REQUEST

- Write a clear and descriptive title and summary.
- Explain what you changed and why.
- Link the ticket or task.
- Request review from the right people.
- Be open to feedback.

"A good pull request shows your work and your thinking."

9 DOCUMENT WHAT CHANGED

- Update relevant documentation (README, runbooks, diagrams, etc.).
- Include any configuration changes.
- Mention required follow-up steps (if any).
- Help the next engineer understand your change.

"If it's not documented, it's like it never happened."

10 VERIFY AFTER DEPLOYMENT

- Check that the change is deployed successfully.
- Validate that the system works as expected.
- Monitor logs, metrics, and dashboards.
- Confirm with the requester or users.

"A change is not complete until it works in production."

11 ASK QUESTIONS BEFORE ASSUMPTIONS BECOME CHANGES

- If you are unsure, ask your team.
- Confirm requirements, impact, and approach.
- It's better to ask early than to fix problems later.
- Curiosity and communication are key DevOps skills.

"Ask. Clarify. Confirm. Then change."

JUNIOR ENGINEER TIP

Take your time, especially in the beginning.
A good engineer is not the one who changes things the fastest, but the one who makes the right changes, safely.

COMMON MISTAKES TO AVOID

- Making changes without understanding
- Ignoring dependencies
- Skipping tests
- Changing production without review
- Poor or no documentation
- Assuming instead of asking

GOOD PROCESS TODAY BUILDS A STRONGER YOU TOMORROW.

VERIQTA

Learn
Practice
Deploy
Grow

PAGE 8

Same
Problems
Stronger
Engineers

CI/CD PIPELINES

FOLLOW THE CODE TO PRODUCTION

AUTOMATE · TEST · DEPLOY · DELIVER · REPEAT

1 WHAT HAPPENS AFTER GIT PUSH?

When you push code to the repository, a CI/CD pipeline automatically runs a series of steps to build, test, package, and deploy your application.

1. CODE PUSH (Git)	2. BUILD	3. TEST	4. SECURITY SCAN	5. PACKAGE	6. STORE	7. DEPLOY	8. VERIFY
Developer pushes code to the repository	Builds the application (e.g. compile, dependencies)	Runs automated tests (unit, integration, etc.)	Checks for vulnerabilities (dependencies, container, code)	Creates an artifact or container image (e.g. JAR, WAR, Docker image)	Pushes to artifact repository or container registry (e.g. Nexus, ECR, Docker Hub)	Deploys to environment (dev, test, prod)	Checks that the application is running correctly

2 BUILD

- Installs dependencies
- Compiles the code
- Fails if there are build errors

Common Tools:
Maven, Gradle, npm

3 TEST

- Runs unit and integration tests
- Checks code quality
- Fails if tests do not pass

Common Tools:
JUnit, PyTest, Jest, SonarQube

4 SECURITY SCAN

- Scans code and dependencies for vulnerabilities
- Checks container images
- Fails if high-risk issues are found

Common Tools:
SonarQube, Trivy, Snyk, OWASP

5 PACKAGE

- Creates a deployable artifact
- Examples: JAR, WAR, ZIP, Docker image

Common Tools:
Maven, Gradle, Docker

6 ARTIFACT OR IMAGE

- Stores the built artifact
- Used later for deployment
- Keeps version history

Common Tools:
Nexus, JFrog Artifactory, AWS ECR, Docker Hub

7 DEPLOY

- Deploys the artifact to an environment
- Can be automatic or manual
- Uses tools like Kubernetes, VMs or cloud services

Common Tools:
Kubernetes, Helm, AWS, Azure, Jenkins

8 VERIFY

- Checks if the application is running
- Runs health checks
- Confirms the deployment was successful

Common Tools:
kubectl, curl, automated tests, monitoring tools

9 WHY PIPELINES FAIL

- Build errors
- Test failures
- Security scan issues
- Configuration mistakes
- Missing environment variables
- Deployment failures
- Infrastructure issues
- Insufficient permissions

10 HOW TO READ PIPELINE LOGS

- Read from the top
- Look for the first error
- Check the specific job that failed
- Read the error message carefully
- Look at stack traces or logs
- Compare with previous successful runs
- Check environment variables and secrets

11 DO NOT REPEAT FAILING JOBS

- Do not keep rerunning a failing pipeline without investigating.
- Rerunning does not fix the root cause.
- It wastes resources and may hide real problems.
- Read the logs, understand the error, fix the issue, then run again.

JUNIOR ENGINEER TIP

A CI/CD pipeline is your friend. It gives you fast feedback, catches mistakes early, and helps you ship reliable software. Learn to read the logs — they will tell you everything.

Same Problems Stronger Engineers

DOCKER SURVIVAL

WHEN "IT WORKS ON MY MACHINE" IS NOT ENOUGH

BUILD • PACKAGE • RUN • SHIP • REPEAT

1 IMAGE vs CONTAINER

IMAGE
- Read-only template
- Contains application
- Used to create containers
- Stored in a registry

CONTAINER
- Running instance of an image
- Has its own filesystem
- Can be started, stopped, deleted
- Ephemeral (can be recreated)

An image is the blueprint.
A container is the running application.

2 DOCKERFILE

- Instructions to build an image
- Defines base image, dependencies, and commands

Example:

```
FROM nginx:1.25
COPY . /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

3 REGISTRY

- Stores Docker images
- Public: Docker Hub
- Private: AWS ECR, GitHub Container Registry, GitLab, etc.

Docker Hub | ECR | GitHub Container Registry

4 PORTS

- Expose container ports
- Map to host ports
- Format: -p host:container

Example:

```
docker run -p 8080:80 nginx
```

- 8080 = host port
- 80 = container port

5 ENVIRONMENT VARIABLES

- Pass configuration to containers
- Keep sensitive values out of the image

Example:

```
docker run -e APP_ENV=prod \
-e DB_HOST=database nginx
```

6 VOLUMES

- Persist data outside the container
- Useful for databases, logs, and uploads

Example:

```
docker run -v /host/data:/app/data
nginx
```

7 LOGS

- View application output
- Useful for troubleshooting

Example:

```
docker logs <container_id>
docker logs -f <container_id>
```

8 ESSENTIAL DOCKER COMMANDS

```
docker ps                  # List running containers
docker ps -a               # List all containers
docker logs <id>           # View logs
docker inspect <id>        # Detailed information
docker exec -it <id> /bin/bash   # Access container
```

9 DOCKER EXEC

- Run commands inside a running container
- Useful for debugging

Example:

```
docker exec -it <container_id>
/bin/bash
docker exec <id> ls /app
```

10 EXIT CODES

- 0 = Success
- Non-zero = Error
- Common codes:
  - 1 = General error
  - 126 = Permission denied
  - 127 = Command not found
  - 137 = Out of memory (OOMKilled)

11 DIAGNOSING A CONTAINER THAT KEEPS EXITING

1. Check the container status (docker ps -a)
2. Look at the logs (docker logs <id>)
3. Check the exit code (docker inspect <id>)
4. Verify the command and arguments
5. Check environment variables and configuration
6. Ensure required files, ports, or dependencies exist
7. Try running the container interactively (docker run -it ...)
8. Fix the issue and test again

JUNIOR ENGINEER TIP
Containers will fail. It's normal. Don't panic.
Read the logs, understand the error, fix the cause.
Practice makes you better.

COMMON MISTAKE
Don't keep rerunning a failing container without investigating. This wastes time and hides the real problem.

VERIQTA

LEARN | PRACTICE | DEPLOY | GROW

Same Problems Stronger Engineers

Kubernetes Today Better Engineers Tomorrow

KUBERNETES SURVIVAL

KNOW WHAT YOU ARE LOOKING AT

UNDERSTAND THE BUILDING BLOCKS. TROUBLESHOOT WITH CONFIDENCE. SHIP TO PRODUCTION.

1 CLUSTER

- A group of nodes that run containerized applications.
- Manages and orchestrates everything.

2 NODE

- A worker machine (virtual or physical) that runs pods.
- Can be a VM or cloud instance (e.g. EC2).

3 NAMESPACE

- Logical separation of resources in a cluster.
- Used to organize environments (dev, test, prod).

4 POD

- The smallest deployable unit in Kuberentes.
- Runs one or more containers.
- Has its own IP address.

5 DEPLOYMENT

- Manages the desired state of your application.
- Creates and updates ReplicaSets.
- Handles rolling updates and rollbacks.

6 REPLICASET

- Maintains a stable set of pod replicas.
- Ensures the desired number of pods are running.
- Usually created and managed by a Deployment.

7 SERVICE

- Provides a stable network endpoint to access pods.
- Load balances traffic to pods.
- Types: ClusterIP, NodePort, LoadBalancer.

8 CONFIGMAP

- Stores non-sensitive configuration data as key-value pairs.
- Used for application configuration (e.g. environment variables, config files).

9 SECRET

- Stores sensitive data (passwords, API keys, tokens).
- Data is base64 encoded.
- Use Kubernetes Secrets or external secret managers (e.g. AWS Secrets Manager).

10 INGRESS

- Manages external access to services (HTTP/HTTPS).
- Provides routing rules (host, path).
- Usually works with a controller (e.g. NGINX Ingress).

11 KUBECTL GET

- Lists Kubernetes resources.

Examples:

```
kubectl get pods
kubectl get deployments
kubectl get services
kubectl get all -n <namespace>
```

12 KUBECTL DESCRIBE

- Shows detailed information about a resource.

Examples:

```
kubectl describe pod <pod-name>
kubectl describe deployment <name>
kubectl describe service <name>
```

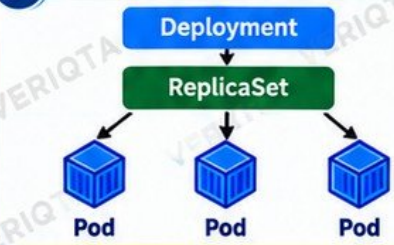
13 KUBECTL LOGS

- Shows logs from a container in a pod.

Examples:

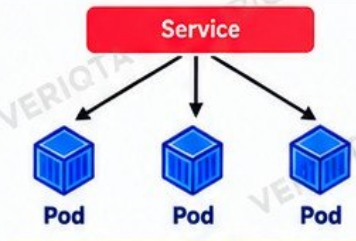
```
kubectl logs <pod-name>
kubectl logs -f <pod-name>
kubectl logs <pod-name> -c <container>
```

14 JUNIOR MENTAL MODEL



Deployment → ReplicaSet → Pods

A Deployment creates and manages a ReplicaSet, which maintains the desired number of Pods.



Service → Pods

A Service provides a stable endpoint and load balances traffic to Pods.

JUNIOR ENGINEER TIP

You don't need to memorize everything. Focus on understanding how the pieces fit together. Use kubectl, explore the resources, and practice in a real or local cluster.

COMMON MISTAKES

- Confusing Deployments and Pods
- Deleting the wrong namespace
- Exposing services incorrectly
- Not checking logs when things fail
- Making changes without understanding the impact

VERIQTA

LEARN | PRACTICE | DEPLOY | GROW

Same Problems Stronger Engineers

Debug Learn Improve Repeat

WHEN KUBERNETES SAYS SOMETHING IS WRONG

COMMON POD ISSUES, HOW TO DIAGNOSE THEM, AND HOW TO FIX THEM

1 PENDING

- Pod is created but not scheduled to a node.
- Usually caused by insufficient resources, node constraints, or scheduling rules.

Common Fixes:
- Check node resources
- Check node selectors, taints and tolerations
- Verify resource requests

2 CRASHLOOPBACKOFF

- Container starts, crashes, and keeps restarting.
- Often caused by application errors, misconfiguration, or missing dependencies.

Common Fixes:
- Check container logs
- Fix application errors
- Verify environment variables and configuration
- Check resource limits

3 IMAGEPULLBACKOFF

- Kubernetes cannot pull the container image from the registry.
- Usually caused by wrong image name, missing tag, registry auth issues, or network problems.

Common Fixes:
- Verify image name and tag
- Check registry credentials
- Ensure network access to the registry

4 OOMKILLED

- Container is killed due to out of memory.
- Happens when the container uses more memory than its limit.

Common Fixes:
- Increase memory limits
- Optimize application memory usage
- Check for memory leaks

5 FAILED PROBES

- Liveness, readiness, or startup probe is failing.
- Container may be healthy but not responding as expected.

Common Fixes:
- Check probe configuration
- Verify the application endpoint
- Increase initial delay or timeout

6 SCHEDULING FAILURES

- Pod cannot be scheduled on any node.
- Caused by insufficient resources, node affinity rules, taints, or topology constraints.

Common Fixes:
- Check cluster resources
- Review scheduling events
- Verify node selectors, taints and resource requests

7 KUBECTL GET PODS

- View the status of pods in your namespace.

Example:

```
kubectl get pods
kubectl get pods -o wide
kubectl get pods -n <namespace>
```

8 KUBECTL DESCRIBE POD

- Shows detailed information about the pod, including events.

Example:

```
kubectl describe pod <pod-name>
kubectl describe pod <pod-name> -n <namespace>
```

9 KUBECTL LOGS

- View the logs from a container in a pod.

Example:

```
kubectl logs <pod-name>
kubectl logs <pod-name> -c <container>
kubectl logs -f <pod-name>
```

10 EVENTS

- Shows recent events in the cluster (e.g. scheduling failures, image pull errors, probe failures).

Example:

```
kubectl get events --sort-by=.lastTimestamp
kubectl get events -n <namespace>
kubectl describe pod <pod-name>
```

11 DO NOT DELETE A FAILING POD

- Deleting a pod removes evidence (logs, events, state).
- Always collect information first before taking action.
- Use logs, describe, and events to diagnose the issue.

Collect evidence, understand the cause, then fix the real problem.

JUNIOR ENGINEER TIP

A failing pod is telling you something. Read the status, logs, and events carefully. Do not rush to delete it.

> "INVESTIGATE BEFORE YOU ELIMINATE."

REAL ENGINEERS TROUBLESHOOT. THEY DON'T JUST RESTART.

Plan Design Build Secure Operate

CLOUD INFRASTRUCTURE
WITHOUT CLICKING RANDOM BUTTONS

ONE CLOUD
MANY POSSIBILITIES
SAME PRINCIPLES

UNDERSTAND. PLAN. USE THE RIGHT TOOLS. AVOID COSTLY MISTAKES.

1 COMPUTE

- Virtual machines (e.g. EC2, Azure VMs, Compute Engine)
- Run applications and services
- Choose the right instance type
- Understand auto scaling
- Consider high availability

Examples:
- AWS EC2, Auto Scaling
- Azure Virtual Machines
- GCP Compute Engine

2 NETWORKING

- VPC / Virtual Network
- Subnets (public and private)
- Route tables
- Internet gateway / NAT gateway
- Security groups / Network ACLs
- DNS and private networking

Examples:
- AWS VPC
- Azure Virtual Network
- GCP VPC

3 STORAGE

- Object storage (e.g. S3)
- Block storage (e.g. EBS)
- File storage (e.g. EFS)
- Lifecycle and backup
- Encryption
- Choose the right storage for your use case

Examples:
- AWS S3, EBS, EFS
- Azure Blob, Disk, Files
- GCP Cloud Storage, Persistent Disk

4 LOAD BALANCERS

- Distributes traffic across multiple instances
- Improves availability and scalability
- Supports health checks
- Works with auto scaling
- Can be internal or external

Examples:
- AWS ELB (ALB/NLB)
- Azure Load Balancer
- GCP Cloud Load Balancing

5 DATABASES

- Managed databases (e.g. RDS, Azure SQL, Cloud SQL)
- Choose the right database type
- Consider backups and replication
- Plan for high availability
- Understand storage, performance, and cost

Examples:
- AWS EDS, Aurora, DynamoDB
- Azure SQL, Cosmos DB
- GCP Cloud SQL, Firestore

6 IAM (IDENTITY AND ACCESS)

- Manages users, roles, and permissions
- Follow least privilege principle
- Use roles instead of long-term keys
- Enable MFA
- Monitor and audit access
- Control access to resources across services

Examples:
- AWS IAM
- Azure Entra ID (Azure AD)
- GCP IAM

7 REGIONS AND AVAILABILITY ZONES

- Regions are separate geographic locations (e.g. us-east-1)
- Availability Zones (AZs) are separate data centers within a region
- Use multiple AZs for high availability
- Consider latency, cost, and compliance requirements

Examples:
- AWS Regions and AZs
- Azure Regions and Availability Zones
- GCP Regions and Zones

8 INFRASTRUCTURE AS CODE

- Provision and manage infrastructure using code
- Repeatable and consistent
- Reduces manual errors
- Enables version control and collaboration
- Examples: Terraform, CloudFormation, Bicep

Benefits:
- Consistency, auditability, faster deployments, lower risk

9 CONSOLE CHANGER AND CONFIGURATION DRIFT

- Making changes directly in the cloud console can create configuration drift.
- Drift happens when the actual infrastructure is different from your Infrastructure as Code (IaC).
- It makes environments inconsistent, harder to manage, and can cause failures.
- Always prefer IaC for production changes.

10 UNDERSTAND BLAST RADIUS

- The blast radius is the scope of impact if something goes wrong.
- Before changing any cloud resource, understand what could be affected.
- Ask: What depends on this resource?
- Consider downtime, data loss, cost impact, and other services.
- Start with small, safe changes when possible.

11 PRODUCTION AWARENESS

"DELETE" MAY REALLY MEAN DELETE.

- Deleting a resource can permanently remove data, services, or access.
- Always double-check what you are deleting.
- Use naming, tagging, and permissions to reduce accidental deletion.
- Be extra careful in production.

JUNIOR ENGINEER TIP

Understand the basics, ask questions, use documentation, and plan before you make changes. Cloud is powerful, but it can also be expensive and unforgiving.

> "THINK BEFORE YOU CLICK. PLAN BEFORE YOU DEPLOY. ENGINEER FOR IMPACT."

REAL ENGINEERS PLAN.

The cloud is not a playground. Understand the impact, use the right tools, and always be intentional with changes.

VERIQTA

LEARN PRACTICE DEPLOY GROW

TERRAFORM AND INFRASTRUCTURE AS CODE SURVIVAL

SAME PROBLEMS BETTER ENGINEERS

Terraform by HashiCorp

WRITE. PLAN. DEPLOY. MANAGE. REPEAT.

Infrastructure as Code (IaC) for real-world DevOps engineers

AUTOMATE INFRASTRUCTURE
REDUCE ERRORS
BUILD REPEATABLE ENVIRONMENTS

1 WHAT IS TERRAFORM?

Terraform is an open source Infrastructure as Code (IaC) tool by HashiCorp. It lets you define, provision, and manage infrastructure using simple configuration files instead of manual clicks in a cloud console.

- Consistent infrastructure
- Repeatable deployments
- Version controlled
- Works across multiple cloud providers

2 CORE CONCEPTS

Configuration
Written in HCL (HashiCorp Configuration Language).

Provider
Plugin that connects Terraform to a cloud platform (AWS, Azure, GCP, etc).

Resource
The infrastructure object you want to create (e.g. EC2, VPC, S3).

Variable
Allows you to input values (e.g. region, instance type).

Output
Displays useful information after deployment (e.g. instance IP).

State
A file that tracks the real-world infrastructure Terraform manages.

3 BASIC TERRAFORM WORKFLOW

Write Configuration (.tf files) → Initialize (install providers) → Plan (see changes) → Apply (create/update infrastructure)

4 KEY TERRAFORM COMMANDS

`terraform fmt`	Formats your configuration files.
`terraform validate`	Checks syntax and configuration for errors.
`terraform plan`	Shows what changes will be made (does not create anything).
`terraform apply`	Creates or updates the infrastructure.

5 WHY ENGINEERS INSPECT THE PLAN

- Shows exactly what will be created, changed, or destroyed.
- Helps you catch mistakes before they affect real systems.
- Lets you review the impact (cost, resources, dependencies).
- Allows team members to approve changes.
- Reduces the risk of accidental deletions.

Always review the plan output. Never skip this step.

6 WHY TERRAFORM STATE MATTERS

- Tracks the real infrastructure Terraform manages.
- Maps your configuration to real resources.
- Helps Terraform know what to create, update, or destroy.
- Must be stored securely (e.g. S3, Azure Storage, GCS).
- State file contains sensitive information.

Do not store state files locally in production.
Use a remote backend and enable state locking.

7 WHY BLINDLY RUNNING APPLY IS DANGEROUS

- Can create, modify, or destroy real infrastructure.
- May lead to accidental data loss.
- Can introduce unexpected cost.
- Might affect other environments or shared resources.
- Changes may be irreversible.
- Can break production systems.

Always understand the changes. Run plan, review, get approval, then apply.

8 JUNIOR ENGINEER TIP

Start with small, safe changes. Read the plan output carefully.
Understand what each resource does. With practice, Terraform becomes predictable and powerful.

Progress Not Perfection

SECRETS, CREDENTIALS, AND ACCESS

Same Problems Stronger Engineers

Security Is Everyone's Responsibility

DO NOT BECOME THE SECURITY INCIDENT

Protect credentials. Protect systems. Protect your career.

1 WHAT ARE SECRETS AND CREDENTIALS?

Secrets are sensitive pieces of information that give access to systems, data, and services. They must be protected at all times.

2 COMMON TYPES

Passwords	API keys	Tokens	SSH keys
User and system passwords	Access to APIs and services	Short-lived or long-lived tokens	Access to servers and repositories

Cloud credentials	Kubernetes Secrets	Environment variables	Secrets managers
AWS, Azure, GCP credentials	Store sensitive data in K8s	Sensitive values in application configs	Tools like AWS Secrets Manager, HashiCorp Vault

3 WHY PROTECTION MATTERS

Exposed credentials can lead to:

- Unauthorised access
- Data breaches
- Financial loss
- Service disruption
- Loss of trust
- Damage to your reputation

4 LEAST PRIVILEGE

Give only the minimum access required to do the job. Do not use admin or root access unless absolutely necessary.

5 MULTI-FACTOR AUTHENTICATION (MFA)

- Use MFA wherever possible.
- It adds an extra layer of security and prevents unauthorized access even if your password is compromised.

6 NEVER SHARE OR PASTE SECRETS

Never paste secrets into:

- Git repositories
- Tickets (e.g. Jira, ServiceNow)
- Screenshots
- Public chat (e.g. Slack, Teams, Discord)
- Emails
- Notion, Confluence, or any shared document

Once a secret is shared, you lose control. It can be copied, forwarded, or leaked.

7 WHAT TO DO IF YOU ACCIDENTALLY EXPOSE A CREDENTIAL

1. Revoke or invalidate the credential immediately.
2. Generate a new credential.
3. Rotate the credential in all systems where it was used.
4. Check for any unauthorized access or suspicious activity.
5. Inform your team and security team immediately.
6. Update any exposed secrets in code, pipelines, or configurations.
7. Document what happened and learn from it.

ACT FAST

- The faster you respond, the lower the risk.
- Be honest and transparent.
- It's better to report a mistake early than to hide it.
- Help improve processes to prevent it from happening again.

> "Security is a habit, not a one-time task."

8 JUNIOR ENGINEER TIP

You do not need to memorize every tool. You need to understand the risks, follow best practices, and know what to do when something goes wrong.

Better Engineers Build Safer Tomorrow

Same Problems Stronger Grow

Learn Practice Deploy Grow

MONITORING AND OBSERVABILITY

LEARN TO SEE PRODUCTION

Better Monitoring Better Products

Metrics. Logs. Traces. A Clearer, Safer, More Reliable System.

1 WHAT IS MONITORING AND OBSERVABILITY?

Monitoring tells you something is wrong, Observability helps you understand why and how to fix it.

2 THE THREE PILLARS

METRICS
Numbers over time (e.g. CPU, memory, requests).

LOGS
Detailed events (e.g. application and system logs).

TRACES
Follow a request across services to find where it is slow or failing.

3 KEY COMPONENTS

DASHBOARDS
Visualize metrics, logs, and traces in one place.

ALERTS
Notify you when something is wrong.

DATA SOURCES
Apps, servers, containers, cloud services, networks, etc.

OBSERVABILITY TOOLS
Prometheus, Grafana, Datadog, Dynatrace, ELK, etc.

4 KEY METRICS TO WATCH

CPU
How much processing is used.

MEMORY
How much memory is used and available.

DISK
Disk usage and I/O performance.

LATENCY
How long requests take to complete.

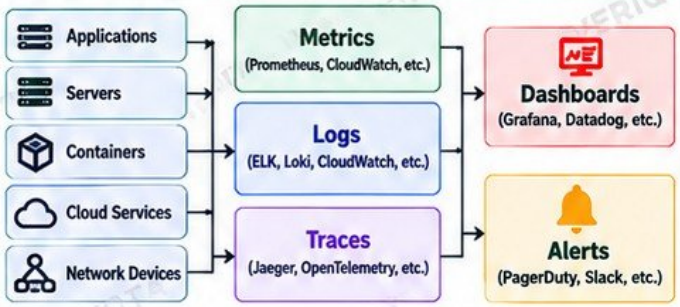
TRAFFIC
Number of requests/users.

ERROR RATE
Percentage of failed requests.

SATURATION
When a resource is close to its limit (e.g. CPU, memory).

BASELINES
Normal behaviour so you can spot abnormal behaviour.

5 HOW IT ALL WORKS TOGETHER



6 WHY ONE RED DASHBOARD DOES NOT AUTOMATICALLY IDENTIFY THE ROOT CAUSE

- A red metric shows a symptom, not the cause.
- Multiple components can affect the same metric.
- The real cause may be in logs or traces.
- It could be an upstream or downstream issue.
- It might be a dependency (database, network, etc.).
- There may be multiple issues happening at the same time.
- You need to investigate, not just react.

A red dashboard tells you something is wrong, not why it is wrong.

7 BEST PRACTICES

- [x] Monitor the right metrics (not everything).
- [x] Set meaningful alerts (avoid alert fatigue).
- [x] Use dashboards for quick context.
- [x] Use logs and traces to investigate.
- [x] Understand normal behaviour (baselines).
- [x] Correlate data across metrics, logs, and traces.
- [x] Document your monitoring and runbooks.
- [x] Continuously improve what you monitor.

8 JUNIOR ENGINEER TIP

When something is wrong: Look at metrics, logs, and traces together. Ask questions, investigate step by step, and avoid assumptions. Observability is a skill that grows with practice.

"More Observability Less Surprises"

OBSERVE
INVESTIGATE
SOLVE
IMPROVE

Learn Practice Deploy Grow

Same Problems Stronger Engineers

Better Deployments Better Engineers

YOUR FIRST PRODUCTION DEPLOYMENT

PLAN. DEPLOY. VERIFY. STAY UNTIL IT'S STABLE.

A successful deployment is more than pressing a button.

1 READ THE CHANGE

- Understand what is being deployed.
- Read the change description and scope.
- Know which services, environments, or configurations are affected.

2 UNDERSTAND DEPENDENCIES

- Identify dependencies (databases, APIs, other services).
- Check if other teams are involved.
- Understand the impact on upstream and downstream systems.

3 CONFIRM APPROVALS

- Ensure all required approvals are in place.
- Follow your organization's change management process.
- Do not deploy without authorization.

4 CHECK SYSTEM HEALTH

- Verify the current health of the system (before the change).
- Check dashboards, metrics, and logs.
- Make sure there are no existing issues that could affect the deployment.

5 KNOW THE DEPLOYMENT PROCEDURE

- Follow the documented deployment steps.
- Use the correct commands, pipelines, or tools (e.g. CI/CD, Helm, Terraform).
- Understand any environment-specific requirements.

6 KNOW THE ROLLBACK PROCEDURE

- Know how to roll back if something goes wrong.
- Identify the rollback method (previous version, Helm rollback, pipeline rollback, etc).
- Test the rollback process in non-production environments when possible.

7 DEPLOY

- Execute the deployment using the approved method.
- Monitor for immediate errors.
- Follow the procedure carefully.

8 WATCH METRICS AND LOGS

- Monitor dashboards, logs, and alerts during and after deployment.
- Look for errors, latency spikes, or unusual behaviour.
- Confirm that the new version is running as expected.

9 VERIFY FUNCTIONALITY

- Test the application and key functionality.
- Confirm that services are working as expected.
- Check both user-facing and backend functionality.

10 DOCUMENT THE RESULT

- Record what was deployed (version, time, etc).
- Document any issues and how they were resolved.
- Update the change ticket or deployment record.
- Share relevant information with the team.

11 NEVER DISAPPEAR IMMEDIATELY AFTER PRESSING DEPLOY

- Stay and monitor until the deployment is stable.
- Be available to respond to any issues.
- Confirm with the team that everything is working.
- Handover properly if you need to leave.

⚠ DEPLOY RESPONSIBLY

"A deployment is not complete until it is working, verified, and the team knows it's stable."

SAME PROBLEMS. BETTER ENGINEERS.

VERIQTA

LEARN | PRACTICE | DEPLOY | GROW

Progress Not Perfection

Same Problems Stronger Engineers

Learn Practice Deploy Grow

YOUR FIRST PRODUCTION INCIDENT

Better Engineers Build Safer Systems

STAY CALM. RESPOND. RESOLVE. LEARN.

Incidents happen. How you respond makes the difference.

1 DO NOT PANIC

- Stay calm and focused.
- Take a deep breath.
- Follow a structured approach.
- Panic leads to mistakes.

2 ESTABLISH IMPACT

- What is affected?
- Who is affected?
- How many users?
- Is it a full outage or partial degradation?
- What is the business impact?

3 CHECK WHAT CHANGED

- Look for recent deployments.
- Check configuration changes.
- Review infrastructure changes.
- Look at recent commits, releases, or maintenance activities.

4 GATHER EVIDENCE

- Collect logs, metrics, and events.
- Check system state.
- Take screenshots if needed.
- Note timestamps.
- Do not delete anything yet.

5 READ ALERTS AND LOGS

- Check monitoring alerts.
- Review relevant logs (application, system, infrastructure).
- Look for error messages, patterns, and timestamps.
- Correlate metrics and logs.

6 COMMUNICATE CLEARLY

- Share a clear and honest status.
- Use simple language.
- Keep stakeholders updated regularly.
- Avoid speculation.
- Document key decisions and actions.

7 ESCALATE WHEN NECESSARY

- Escalate early if you need help.
- Involve the right people (senior engineers, platform team, vendor, etc.).
- Do not try to handle everything alone.

8 MITIGATE BEFORE PERFECT DIAGNOSIS WHEN IMPACT DEMANDS IT

- If the service is severely impacted, take safe mitigation steps.
- You can stabilize the service first, then continue investigating.
- Examples: scale, restart, failover, rollback, block traffic.
- Communicate what you are doing and why.

9 VERIFY RECOVERY

- Confirm that the service is working.
- Check metrics, logs, and user feedback.
- Ensure no other components are still affected.
- Monitor for a reasonable period of time.

10 PRESERVE EVIDENCE

- Keep logs, metrics, and timelines.
- Save relevant screenshots and alerts.
- Do not delete resources that help with the investigation.
- You may need this for the post-incident review.

11 PARTICIPATE IN THE POST-INCIDENT REVIEW

- Discuss what happened.
- Identify the root cause.
- Share what went well and what can be improved.
- Agree on action items.
- Help implement safeguards to prevent recurrence.

KEY MINDSET

- [x] Incidents are learning opportunities.
- [x] Be honest, transparent, and accountable.
- [x] Focus on the system, not blame.
- [x] Help improve the process.
- [x] You are part of the solution.

TROUBLESHOOTING MODEL

Symptom (What is the issue?) → **Evidence** (What do you see?) → **Hypothesis** (What do you think is causing it?) → **Test** (How do you verify?) → **Cause** (What is the real reason?) → **Fix** (What is the solution?) → **Verify** (Is it resolved?)

JUNIOR ENGINEER TIP

You won't always get it right, and that's normal. What matters is that you stay calm, follow a process, communicate well, and keep learning.

Progress Not Perfection

Learn Practice Deploy Grow

ON-CALL SURVIVAL

WHAT TO DO WHEN THE ALERT FIRES

Better On-Call Better Engineers

Stay calm. Follow a process. Solve the problem. Keep the service running.

1 READ THE ALERT CAREFULLY

- Understand what is alerting.
- Read the full message.
- Check the time, service, and environment.
- Look for clues (error, latency, CPU, etc.).

2 DETERMINE SEVERITY

P1	Service down / major impact Act immediately
P2	Significant degradation Investigate quickly
P3	Minor issue / limited impact Investigate
P4	Informational Monitor

3 CHECK DASHBOARDS

- Look at key metrics (CPU, memory, latency, traffic, error rate).
- Compare with normal baselines.
- Check if the issue is isolated or widespread.

4 OPEN THE RUNBOOK

- Find the runbook for the alert or service.
- Follow the documented troubleshooting steps.
- Check known issues and previous incidents.

5 LOOK FOR RECENT CHANGES

- Check recent deployments.
- Look for configuration changes.
- Review infrastructure changes.
- Check for database, network, or third-party changes.

6 CHECK LOGS AND SYSTEM STATE

- Review application logs.
- Check system and infrastructure logs.
- Look for errors, warnings, and anomalies.
- Verify the state of key components (pods, services, databases, etc.).

7 AVOID RANDOM FIXES

- Do not make uninformed changes.
- Avoid repeatedly restarting services.
- Do not guess or try "just to see."
- Follow a structured troubleshooting approach.

8 ESCALATE EARLY WHEN NECESSARY

- Escalate if you need help or lack the required access.
- Involve the right people (senior engineer, platform team, vendor, etc.).
- Escalation is a sign of good judgement, not failure.

9 RECORD ACTIONS

- Keep a clear timeline of what you did.
- Record commands, changes, and observations.
- This helps with handover and the post-incident review.

10 CONFIRM SERVICE RECOVERY

- Verify that the issue is fully resolved.
- Check metrics, logs, and user feedback.
- Ensure no related components are still affected.
- Monitor for a reasonable period before closing.

11 HANDOVER UNRESOLVED ISSUES PROPERLY

- Document the current status.
- Share what has been done and what is still outstanding.
- Provide useful context, logs, and links.
- Ensure the next engineer can continue without starting from zero.

JUNIOR ENGINEER TIP

"Escalation is part of engineering, not failure."

Ask for help, share what you know, and work as a team.

ON-CALL MINDSET

- ☑ Stay calm and objective.
- ☑ Think about impact, not just the alert.
- ☑ Communicate clearly and early.
- ☑ Use the runbook and follow the process.
- ☑ Be honest about what you know and don't know.
- ☑ Focus on restoring service safely.
- ☑ Keep learning from every incident.
- ☑ You are part of a team.

Calm Engineers Solve Problems. Panic Creates More Problems.

SAME PROBLEMS. BETTER ENGINEERS.

VERIQTA

LEARN | PRACTICE | DEPLOY | GROW

Progress Not Perfection

PRODUCTION MISTAKES, RECOVERY, AND PROFESSIONAL COMMUNICATION

Mistakes Happen Better Engineers Learn

Same Problems Stronger Engineers

**MISTAKES ARE PART OF THE JOURNEY.
HOW YOU HANDLE THEM DEFINES YOUR GROWTH.**

1 COMMON PRODUCTION MISTAKES

You ran the wrong command
e.g. deleted a resource, scaled to 0, changed permissions.

You deployed the wrong configuration
e.g. incorrect values, image tag, environment variables.

You changed the wrong environment
e.g. ran a production command in staging or vice versa.

A deployment caused an outage
e.g. application not starting, high error rate, database issue, service unavailable.

2 WHAT TO DO IMMEDIATELY

Stop making additional uncontrolled changes
Do not make things worse.

Tell the team quickly
Communicate early, even if you don't have all the details.

State exactly what changed
Be clear and honest. Include commands, configurations, or actions.

Help restore service
Follow runbooks, work with the team, and focus on recovery.

3 PROFESSIONAL COMMUNICATION

- Be honest and transparent
- Communicate clearly and calmly
- Share facts, not assumptions
- Keep the team updated
- Ask for help when needed
- Use the right channels (Slack, Teams, incident bridge, etc.)

> "Early communication builds trust. Silence creates bigger problems."

4 IMPORTANT REMINDERS

Do not hide evidence
Be open about what happened.

Document the timeline
Include what you did, when it happened, and what the impact was.

Learn through blameless review
Focus on what happened and how to prevent it, not who to blame.

Build safeguards so the same failure is harder to repeat
Automate checks, improve processes, and add validations.

5 INCIDENT RECOVERY FLOW

IDENTIFY	STOP	COMMUNICATE	RECOVER	REVIEW
What happened?	Prevent further impact	Inform the team	Restore service	Learn and improve

Fast action. Clear communication. Stronger systems.

6 JUNIOR ENGINEER TIP

Mistakes do not make you a bad engineer. Hiding them, not learning from them, and repeating them does.

A real engineer takes responsibility, learns, and helps make the system stronger.

Progress Over Perfection

JUNIOR DEVOPS ENGINEER PRODUCTION SURVIVAL LAB

FROM THEORY TO REAL-WORLD PRACTICE

Same Problems Stronger Engineers

Practice Build Confidence Grow

Follow a complete hands-on workflow and experience the real DevOps journey from ticket to a successful production deployment (and recovery).

1 LAB STEPS

1	**Receive a realistic deployment ticket**	Understand the requirement, scope, and acceptance criteria.
2	**Locate the repository**	Find the correct codebase and related documentation.
3	**Create a Git branch**	Use a meaningful branch name for your changes.
4	**Inspect the application**	Understand the code, configuration, and dependencies.
5	**Build the container**	Build and test the Docker image locally.
6	**Review infrastructure configuration**	Check Terraform, Helm, or other IaC files.
7	**Deploy to Kubernetes**	Use Helm or kubectl to deploy to the target environment.
8	**Verify Pods and Services**	Confirm everything is running as expected.
9	**Inspect logs and metrics**	Check logs, metrics, and dashboards for issues.
10	**Introduce a controlled failure**	Make a deliberate change (e.g. wrong image tag).
11	**Diagnose the symptoms**	Use logs, events, and metrics to investigate.
12	**Identify the root cause**	Find and confirm the actual cause of the failure.
13	**Apply or roll back the fix**	Fix the issue or roll back to the previous version.
14	**Verify recovery**	Ensure the application is healthy again.
15	**Document what happened**	Capture the timeline, cause, and resolution.
16	**Write a short handover**	Share key details for the team.

2 FINAL SURVIVAL WORKFLOW

Practice Real Scenarios Build Real Skills

3 JUNIOR ENGINEER TIP

Real learning happens when things go wrong. Safe practice today builds confidence for real production tomorrow.

> **Mistakes in a lab make you a better engineer in production.**